УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

# ЛАБОРАТОРНАЯ РАБОТА № 20

## УСТАНОВКА И НАСТРОЙКА ФРЕЙМВОРКА ИЛИ БИБЛИОТЕКИ НА ПК. СОЗДАНИЕ БАЗОВОГО ПРОЕКТА. ЗАПУСК ПРИЛОЖЕНИЯ

## МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

## 1 Цель работы

1.1 Установка и настройка фреймворка или библиотеки на ПК. Создание базового проекта. Запуск приложения

## 2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

## 3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

## 4 Основные теоретические положения

### 4.1 Что такое Django

Django — это фреймворк для создания веб-приложений с помощью языка программирования Python.

Django был создан в 2005 году, когда веб-разработчики из газеты Lawrence Journal-World стали использовать Python в качестве языка для создания веб-сайтов. А в 2008 году вышел публичный первый релиз фреймворка. На сегодняшний день он продолжает развиваться. Так, текущей версией фреймворка на момент написания этой статьи является версия 6.0, которая вышла в декабре 2025 года. Но кроме основной версии регулярно выходят подверсии, а также обновления и исправления в безопасности.

Django довольно популярен. Он используется на многих сайтах, в том числе таких, как Pinterest, PBS, Instagram, BitBucket, Washington Times, Mozilla и многих других.

Фреймворк является бесплатным. Он развивается как open source, его исходный код открыт, его можно найти репозитории на githube.

На Django можно создавать широкий диапазон веб-приложений: от небольших персональных сайтов до высоконагруженных сложных веб-сервисов.

Django по умолчанию предлагает готовую функциональность для ряда распространенных задач, например, систему аутентификации, генерацию карт сайта и т.д., благодаря чему нам можно не изобретать велосипед и достаточно взять уже готовые компоненты.

В Django большое внимание уделяется безопасности, благодаря чему фреймворк помогает разработчикам избежать многих распространенных проблем в системе безопасности, например, sql-инъекций.

Фреймворк Django реализует архитектурный паттерн Model-View-Template или сокращенно MVT, который по факту является модификацией распростаненного в веб-программировании паттерна MVC (Model-View-Controller).

Схематично мы можем представить архитектуру MVT в Django следующим образом (рисунок 1):

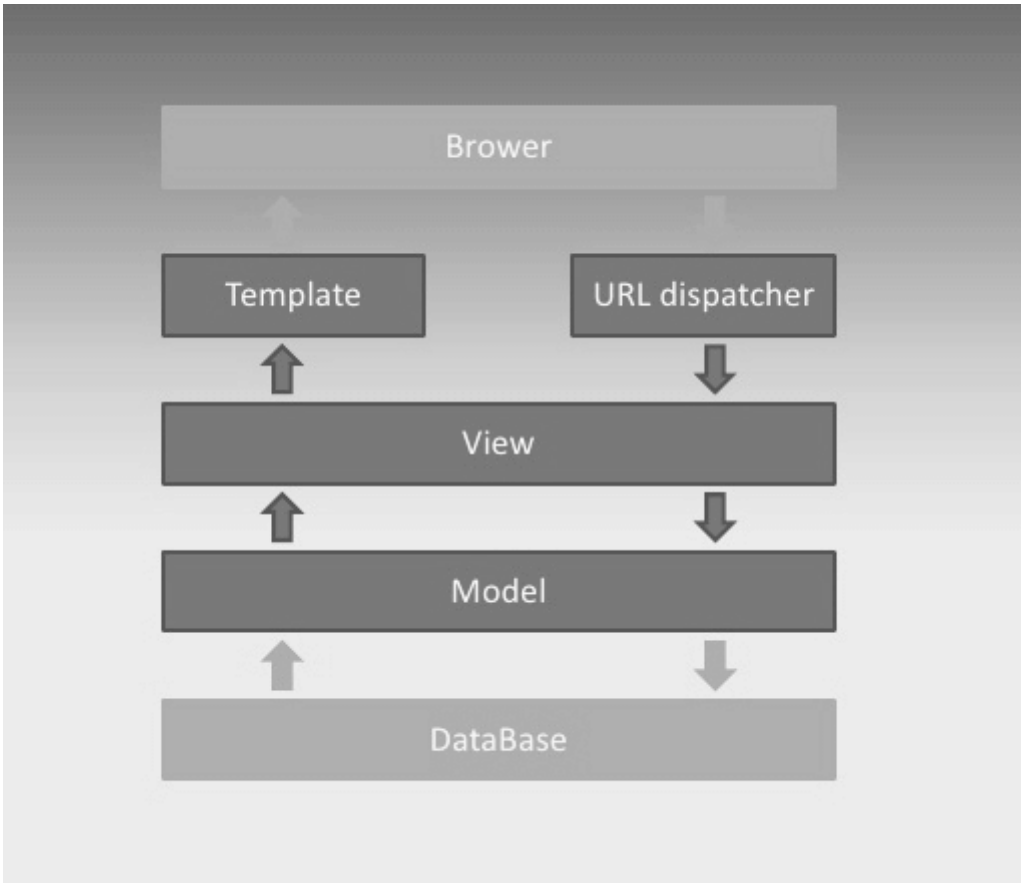



Рисунок 1 - Архитектура MVT в Django

Основные элементы паттерна:

– URL dispatcher: при получении запроса на основании запрошенного адреса URL определяет, какой ресурс должен обрабатывать данный запрос;

– View: получает запрос, обрабатывает его и отправляет в ответ пользователю некоторый ответ. Если для обработки запроса необходимо обращение к модели и базе данных, то View взаимодействует с ними. Для создания ответа может применять Template или шаблоны. В архитектуре MVC этому компоненту соответствуют контроллеры (но не представления);

– Model: описывает данные, используемые в приложении. Отдельные классы, как правило, соответствуют таблицам в базе данных;

– Template: представляет логику представления в виде сгенерированной разметки html. В MVC этому компоненту соответствует View, то есть представления.

Когда к приложению приходит запрос, то URL dispatcher определяет, с каким ресурсом сопоставляется данный запрос и передает этот запрос выбранному ресурсу. Ресурс фактически представляет функцию или View, который получает запрос и определенным образом обрабатывает его. В процессе обработки View может обращаться к моделям и базе данных, получать из нее данные, или, наоборот, сохранять в нее данные. Результат обработки запроса отправляется обратно, и этот результат пользователь видит в своем браузере. Как правило, результат обработки запроса представляет сгенерированный html-код, для генерации которого применяются шаблоны (Template).

### 4.2 Установка и настройка Django

Существуют разные способы установки Django. Рассмотрим рекомендуемый способ.

Пакеты Django размещаются в центральном репозитории для большинства пакетов Python - Package Index (PyPI). И для установки из этого репозитория нам потребуется пакетный менеджер pip. Менеджер pip позволяет загружать пакеты и управлять ими. Обычно при установке python также устанавливается и менеджер pip. В этом случае мы можем проверить версию менеджера, выполнив в командной строке/терминале команду pip -V (V - с заглавной буквы):

C:\Users\eugen>pip -V

pip 23.3.1 from C:\Users\eugen\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip (python 3.12)

C:\Users\eugen>

Если pip не установлен, то мы увидим ошибку типа

"pip" не является внутренней или внешней командой, исполняемой программой или пакетным файлом

В этом случае нам надо установить pip. Для этого можно выполнить в командной строке/консоли следующую команду:

python3 -m ensurepip --upgrade

Если pip ранее уже был установлен, то можно его обновить с помощью команды

python3 -m pip install --upgrade pip

Виртуальная среда или venv не является неотъемлимой частью разработки на Django. Однако ее рекомендуется использовать, так как она позволяет создать множество виртуальных сред Python на одной операционной системе. Благодаря виртуальной среде приложение может запускаться независимо от других приложений на Python.

В принципе можно запускать приложения на Django и без виртуальной среды. В этом случае все пакеты Django устанавливаются глобально. Однако что если после создания первого приложения выйдет новая версия Django? Если мы захотим использовать для второго проекта новую версию Django, то из-за глобальной установки пакетов придется обновлять первый проект, который использует старую версию. Это потребует некоторой дополнительной работы по

обновлению, так как не всегда соблюдается обратная совместимость между пакетами. Если мы решим использовать для второго проекта старую версию, то мы лишиемся потенциальных преимуществ новой версии. И использование виртуальной среды как раз позволяет разграничить пакеты для каждого проекта.

Для работы с виртуальной средой в python применяется встроенный модуль venv.

Итак, создадим вируальную среду. Вначале определим каталог для проектов django. Например, пусть это будет каталог C:\django. Прежде всего перейдем в терминале/командной строке в этот каталог с помощью команды cd.

cd C:\django

Затем для создания виртуальной среды выполним следующую команду:

python3 -m venv .venv

Модулю venv передается название среды, которая в данном случае будет называться ".venv". Для наименования виртуальных сред нет каких-то определенных условностей. Пример консольного вывода:

C:\Users\eugen>cd C:\djangoПереход к папке будущей виртуальной среды

C:\django>python3 -m venv .venvСоздание виртуальной среды

C:\django>

После этого в текущей папке (C:\django) будет создан подкаталог ".venv".

Для использования виртуальную среду надо активировать. И каждый раз, когда мы будем работать с проектом Django, связанную с ним виртуальную среду надо активировать. Например, активируем выше созданную среду, которая располагается в текущем каталоге в папке .venv. Процесс активации немного отличается в зависимости от операционной системы и от того, какие инструменты применяются. Так, в Windows можно использовать командную строку и PowerShell, но между ними есть отличия.

Если наша ОС - Windows, то в папке .venv/Scripts/ мы можем найти файл activate.bat), который активирует виртуальную среду. Так, в Windows активация виртуальной среды в коммандной строке будет выглядеть таким образом:

.venv\Scripts\activate.bat

Также при работе на Windows в папке .venv/Scripts/ мы можем найти файлactivate.ps1, который также активирует виртуальную среду, но применяется только в PowerShell. Но при работе с PowerShell следует учитывать, что по умолчанию в этой оболочке запрещено применять скрипты. Поэтому перед активацией среды необходимо установить разрешения для текущего пользователя. Поэтому для активации виртуальной среды в PowerShell необходимо выполнить две следующих команды:

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

.venv\Scripts\Activate.ps1

Для Linux и MacOS активация будет производиться с помощью следующей команды:

source .venv/bin/activate

Далее я буду приводить примеры на основе командной строки Windows, однако все остальные примеры не будут зависеть от того, что используется - PowerShell или командная строка, Windows, Linux или MacOS. В любом случае

после успешной активации слева от текущего каталога мы увидим в скобках название виртуальной среды:

```
C:\Users\eugen>cd C:\django
C:\django>python3 -m venv .venv
C:\django>.venv\Scripts\activate.batАктивация виртуальной среды
(.venv) C:\django> Виртуальная среда активирована
```

После активации виртуальной среды для установки Django выполним в консоли следующую команду

```
python3 -m pip install Django
```

Она устанавливает последнюю версию Django.

```
(.venv) C:\django>python -m pip install Django
Collecting Django
  Downloading django-6.0-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref>=3.9.1 (from Django)
  Downloading asgiref-3.11.0-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.5.0 (from Django)
  Downloading sqlparse-0.5.4-py3-none-any.whl.metadata (4.7 kB)
Downloading django-6.0-py3-none-any.whl (8.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━ 8.3/8.3 MB 1.5 MB/s eta 0:00:00
Downloading asgiref-3.11.0-py3-none-any.whl (24 kB)
Downloading sqlparse-0.5.4-py3-none-any.whl (45 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━ 45.9/45.9 kB 2.9 MB/s eta 0:00:00
Installing collected packages: sqlparse, asgiref, Django
Successfully installed Django-6.0 asgiref-3.11.0 sqlparse-0.5.4
(.venv) C:\django>
```

Если нам интересует конкретная версия Django, то мы можем указать ее при установке:

```
python3 -m pip install django~=5.0.0
```

Чтобы убедиться, что все установлено правильно, мы можем перейти к интерпретатору python. Для этого введем в терминале команду

```
python3
```

И затем выполним последовательно следующие две инструкции:

```
>>> import django
>>> print(django.get_version())
```

Консольный вывод в моем случае:

```
(.venv) C:\django>python3
Python 3.12.3 (main, Nov  6 2025, 13:44:16) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import django
>>> print(django.get_version())
6.0
>>>
```

После окончания работы с виртуальной средой мы можем ее деактивировать с помощью команды:

Deactivate

### 4.3 Создание первого проекта

При установке Django в папке виртуальной среды устанавливается утилита django-admin. А на Windows также исполняемый файл django-admin.exe. Их можно найти в папке виртуальной среды, в которую производилась установка Django: на Windows - в подкаталоге Scripts, а на Linux/MacOS - в каталоге bin (рисунок 2).

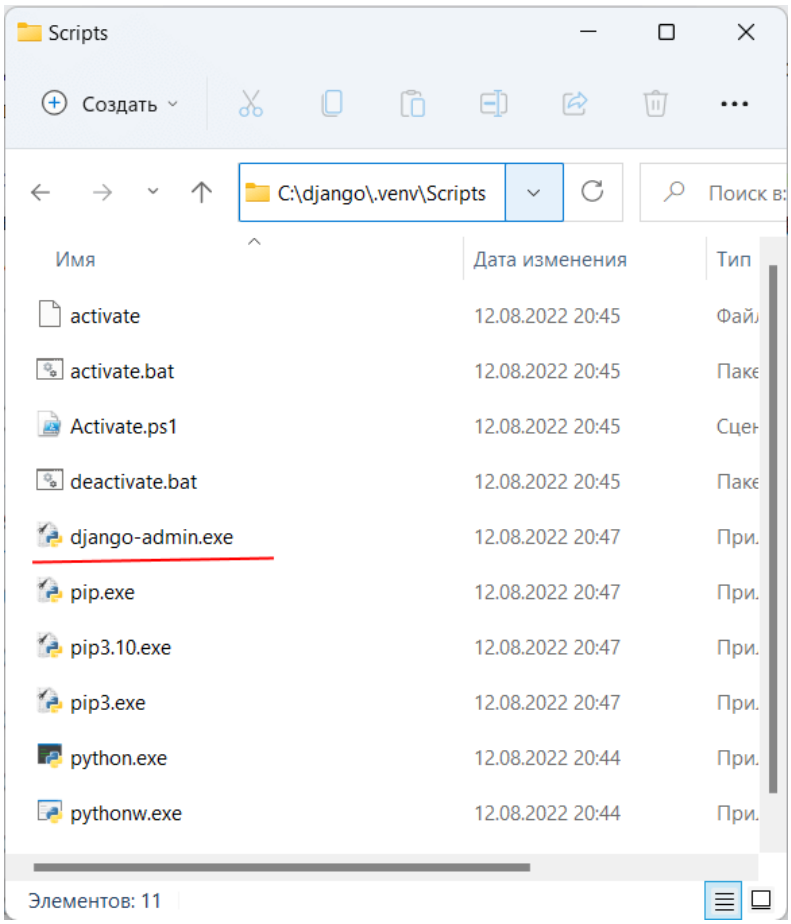



Рисунок 2 - Django-admin в Django и Python

django-admin предоставляет ряд команд для управления проектом Django. В частности, для создания проекта применяется команда startproject. Этой команде в качестве аргумента передается название проекта.

Итак, создадим первый на Django. Пусть он будет располагаться в той же папке, где располагается каталог виртуальной среды. И для этого вначале активируем ранее созданную виртуальную среду (например, среду .venv, которая была создана в прошлой теме, если она ранее не была активирована) (рисунок 3).

И после активации виртуальной среды выполним следующую команду

c:\django>django-admin startproject metanit

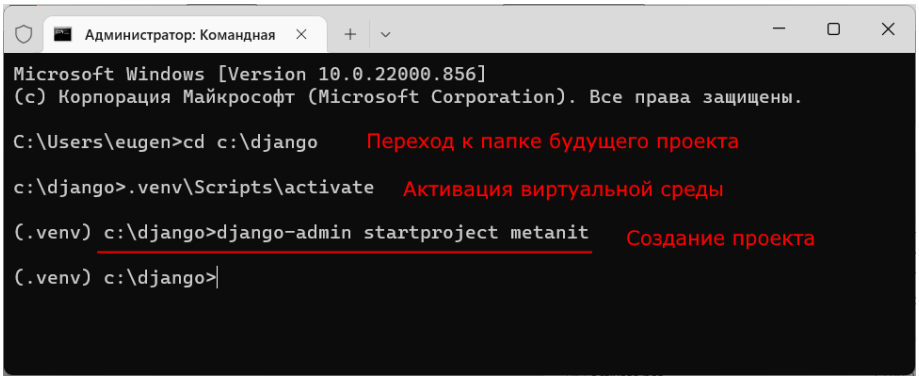



Рисунок 3 - Создание проекта на Python и Django

В данном случае мы создаем проект с именем "metanit". И после выполнения этой команды в текущей папке (c:\django) будет создан каталог metanit (рисунок 4).

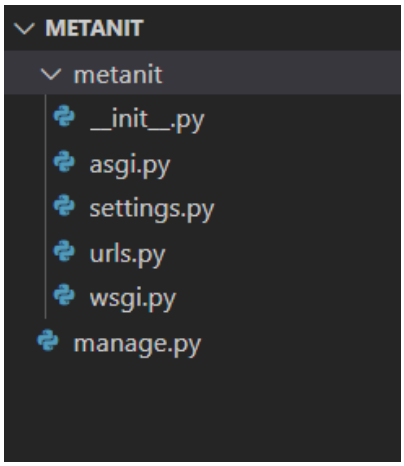



Рисунок 4 - Первый проект на Python и Django

Созданный каталог будет состоять из следующих элементов:

– manage.py: выполняет различные команды проекта, например, создает и запускает приложение

– metanit - собственно папка проекта metanit, которая содержит следующие файлы:

– __init__.py: данный файл указывает, что папка, в которой он находится, будет рассматриваться как модуль. Это стандартный файл для программы на языке Python;

– settings.py: содержит настройки конфигурации проекта;

– urls.py: содержит шаблоны URL-адресов, по сути определяет систему маршрутизации проекта;

– wsgi.py: содержит свойства конфигурации WSGI (Web Server Gateway Inerface). Он используется при развертывании проекта;

– asgi.py: название файла представляет сокращение от Asynchronous Server Gateway Interface и расширяет возможности WSGI, добавляя поддержку для взаимодействия между асинхронными веб-серверами и приложениями.

Запустим проект на выполнение. Для этого с помощью команды cd перейдем в консоли к папке проекта (рисунок 5). И затем для запуска проекта выполним следующую команду:

python manage.py runserver

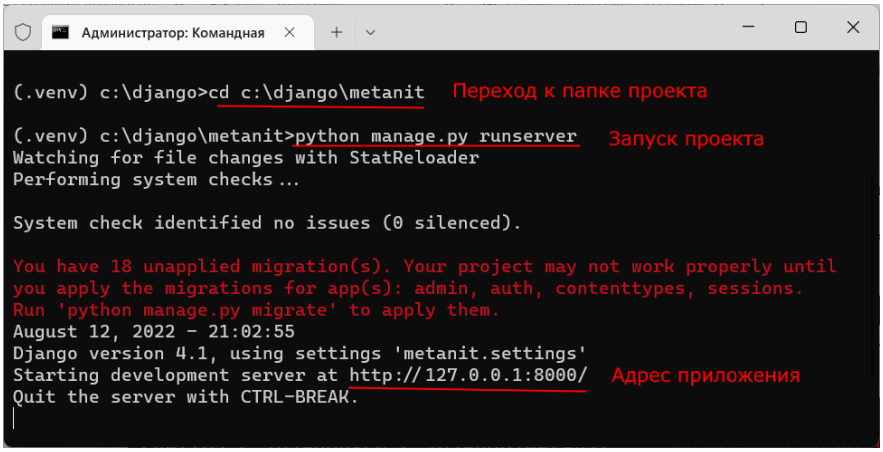



Рисунок 5 - Запуск первого проекта на Python и Django

После запуска проекта в консоли мы увидим адрес, по которому запущен проект. Как правило, это адрес http://127.0.0.1:8000/. Откроем любой веб-браузер и введем данный адрес в адресную строку браузера. И нам откроется содержимое по умолчанию (рисунок 6).

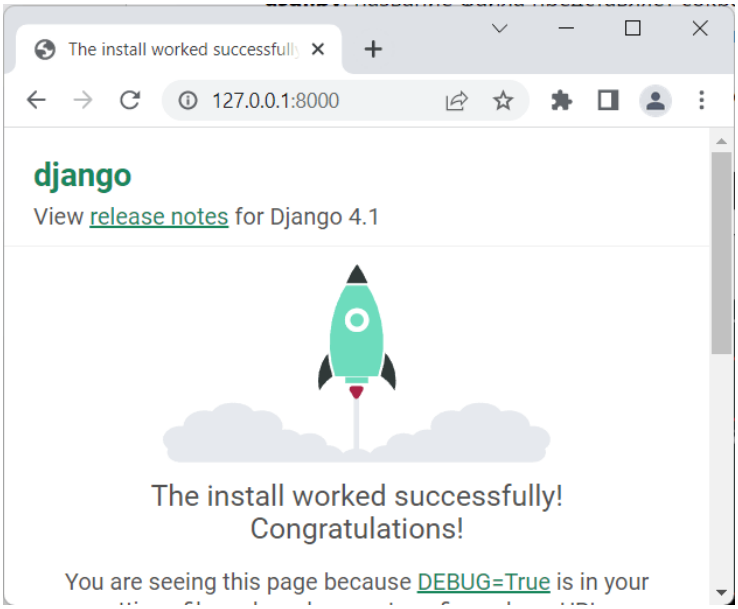



Рисунок 6 - Первое веб-приложение на Python и Django

### 4.4 Структура Django

Фреймворк основывается на четырёх главных компонентах:

– модели (Models) — взаимодействуют с базой данных и достают из неё ту информацию, которую необходимо отобразить в браузере;

– представления (Views) — обрабатывают запрос и обращаются к модели, сообщая ей, какую информацию необходимо достать из базы данных;

– шаблоны (Templates) — показывают, каким именно образом необходимо показать информацию, полученную из базы данных;

– URL-маршруты (URL dispatcher) — перенаправляют HTTP-запрос от браузера в представления.

### 4.5 Практическая часть

1 Установка Visual Studio Code Зайдите на сайт https://code.visualstudio.com/ и скачайте Visual Studio Code;

2 Установка Python Extension Pack:

Заходим в программу VS Code. Слева ищем иконку Расширения (Extensions) и нажимаем. Появляется поисковая строка в ней пишем Python Extension Pack и скачиваем расширение (рисунок 7).

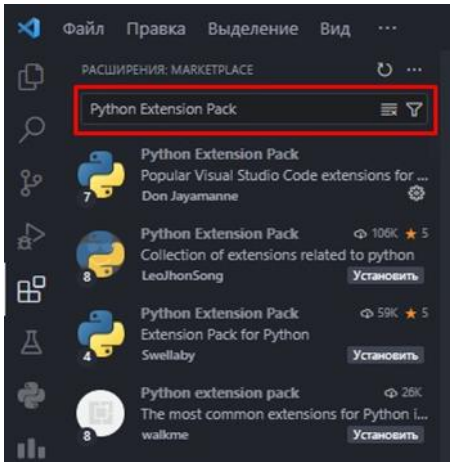



Рисунок 7 - Расширение Python Extension

3 Установка виртуального окружения и Django:

Создайте новую папку на локальном диске и назовите её под вашей Фамилией. В ней создайте папку и назовите её

«myproject». Это будет вашей папкой проекта! В эту папку мы и будем устанавливать наше виртуальное окружение. После создания папок, откроем в VS Code папку нашего проекта. Если вы все сделали правильно, у вас будет открыта папка проекта (рисунок 8).

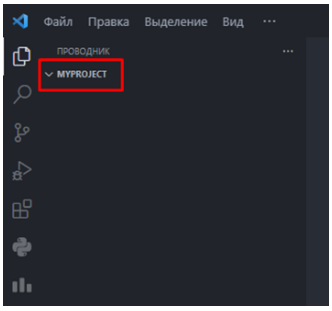



Рисунок 8 - Папка проекта в VS Code

Открываем терминал. Его можно открыть двумя способами:

1 В командном центре сверху найти вкладку «Терминал» и нажать на кнопку «Создать терминал» (рисунок 9);

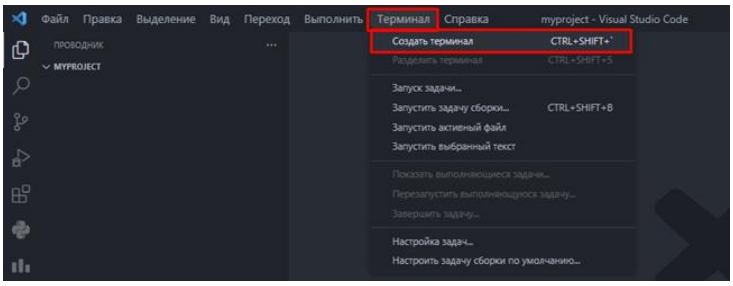



Рисунок 9 - Команда «Создать терминал» в командном центре в VS Code

2 Нажать сочетание клавиш «Ctrl + Shift + `» (` – буква «ё»)

Пишем поочередно список из следующих команд, чтобы создать виртуальное окружение и установить Django (рисунок 10).

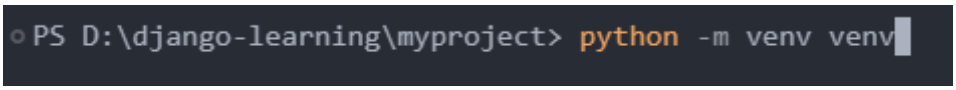



Рисунок 10 - Создание виртуального окружения

У нас появилась папка виртуального окружения в папке проекта (рисунок 11).

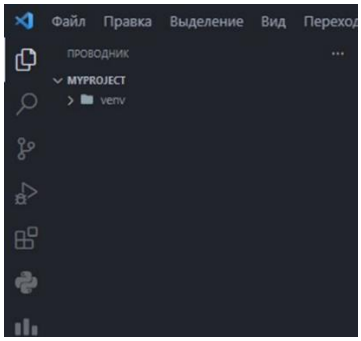



Рисунок 11 - Папка виртуального окружения «venv»

Далее, необходимо активировать виртуальное окружение (рисунок 12).

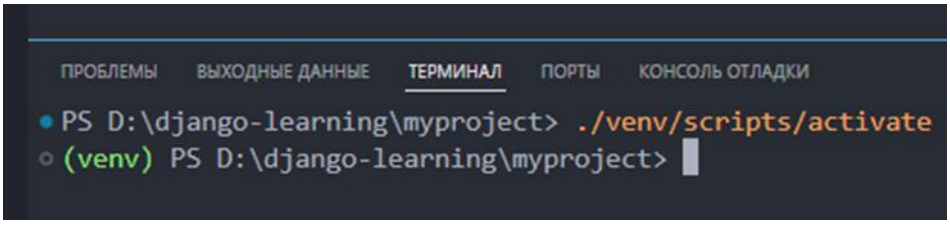



Рисунок 12 - Активация виртуального окружения

Если в терминале появляется предупреждение об отсутствии возможности проверки издателя, и предлагает запустить программу отправляем английскую букву А (рисунок 13).

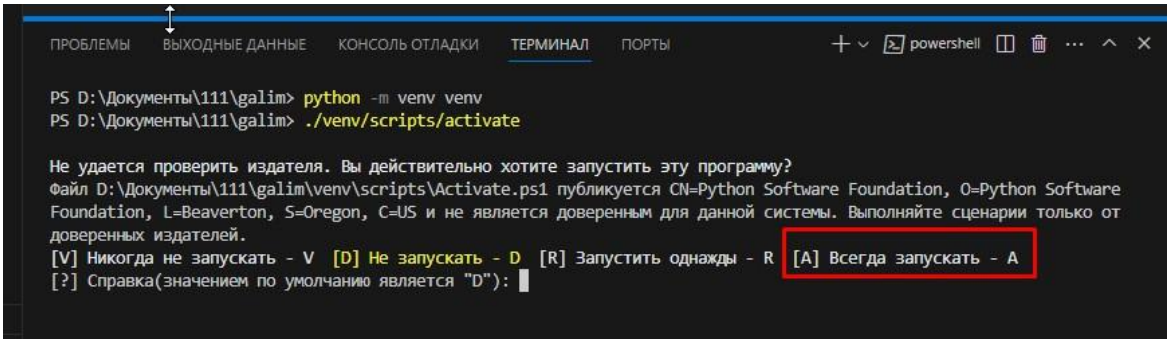



Рисунок 13 - Предупреждение об отсутствии возможности проверки издателя

3 Скачаем Django в папку проекта и немного подождем, пока Django установится (рисунок 14).

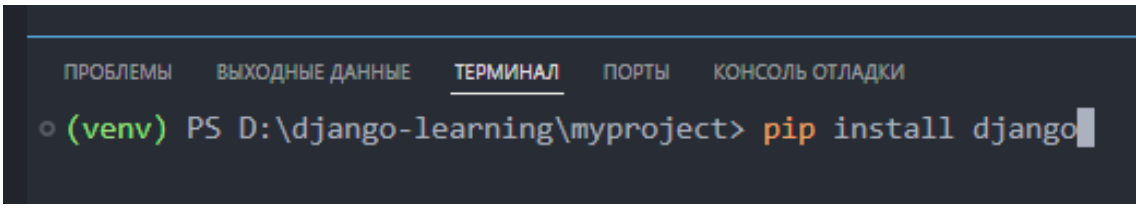



Рисунок 14 - Команда для установки Django

Поздравляю, Django установлен (рисунок 15)!

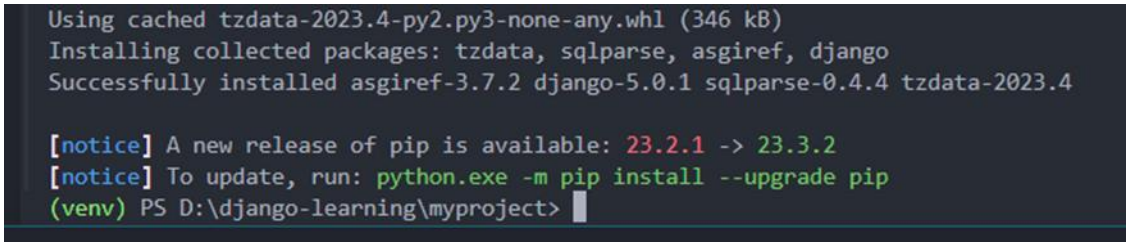



Рисунок 15 - Уведомление об успешной установке Django

Также, VS Code говорит нам, что наша версия pip’а немного устарела, для обновления пропишите команду. Следуем его рекомендациям и прописываем данную команду в терминале (рисунок 16).

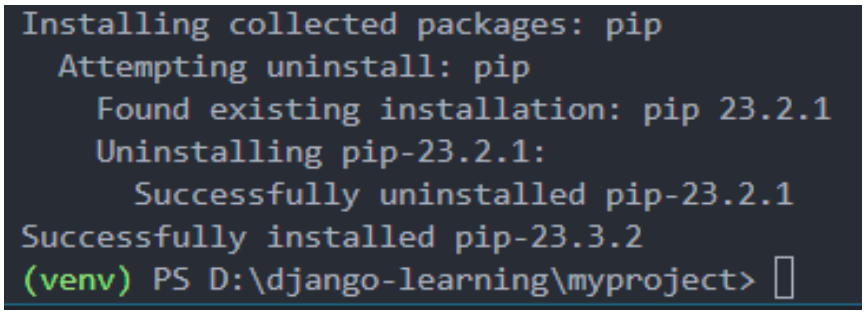



Рисунок 16 - Уведомление об успешном обновлении пакетного менеджера pip

4 Создание нового Django-проекта (рисунок 17).

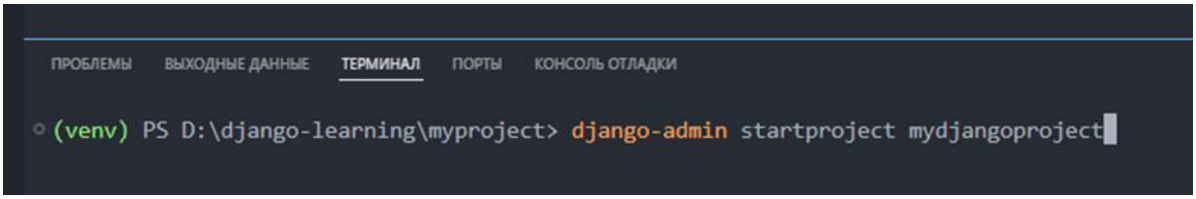



Рисунок 17 - Команда для создания проекта в Django

– **mydjangoproject** – имя проекта. После выполнения этой команды появится **папка Django-проекта mydjangoproject**, в которой будет папка с точно таким же названием, как и папка Django-проекта, со следующей структурой (рисунок 18):

– **manage.py** – управляющий скрипт, который используется для различных административных задач, таких как запуск сервера, создание миграций, управление командами Django и т.д.;

– **init.py** – пустой файл, который говорит Python’у, что mydjangoproject должен рассматриваться как пакет;

– **settings.py** – в этом файле хранятся настройки проекта, такие как БД, настройки безопасности, используемый шаблон и т.д;

– **urls.py** – файл, который содержит ссылки (иными словами, URL-адреса или «маршруты») проекта, определяющие, какие представления должны быть вызваны для определенных URL-адресов;

– **wsgi.py** – файл используется для развертывания проекта на сервере, который поддерживает WSGI (Web Server Gateway Interface).

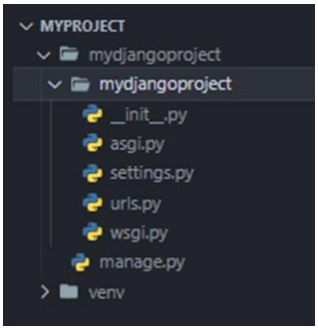



Рисунок 18 - Структура проекта

5 Запуск Django-проекта

Для запуска Django-проекта, перейдем в папку mydjangoproject следующей командой для смены директории (рисунок 19):

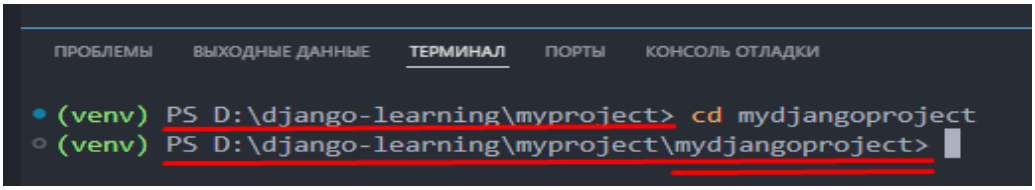



Рисунок 19 - Команда cd для смены директории на уровень выше

Далее, необходим прописать команду для запуска Django-проекта на локальном сервере (рисунок 20).

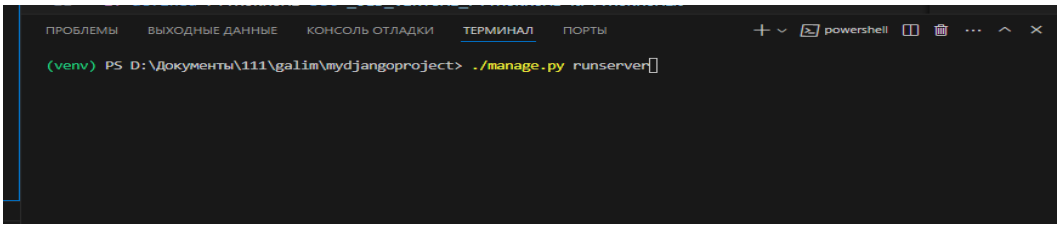



Рисунок 20 - Команда для запуска Django-проекта

Если команда ./manage.py runserver не сработает, напишите полную версию команды (python manage.py runserver) (рисунок 21).

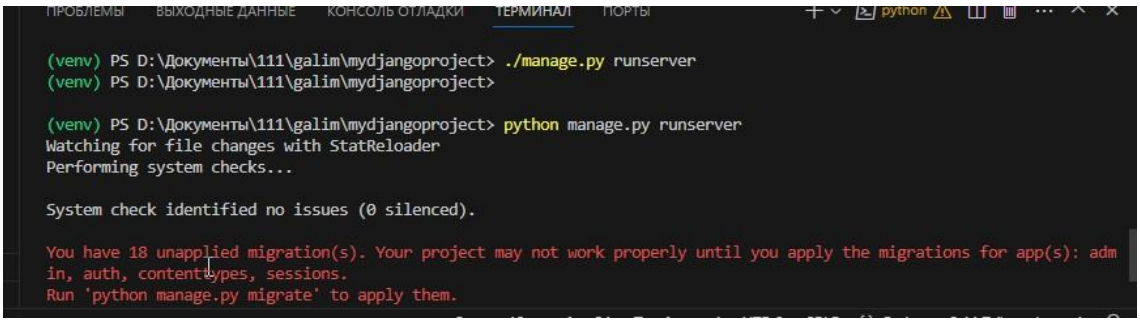



Рисунок 21 - Запуск сервера командой python manage.py runserver

Поздравляю! Мы запустили локальный сервер (рисунок 22).

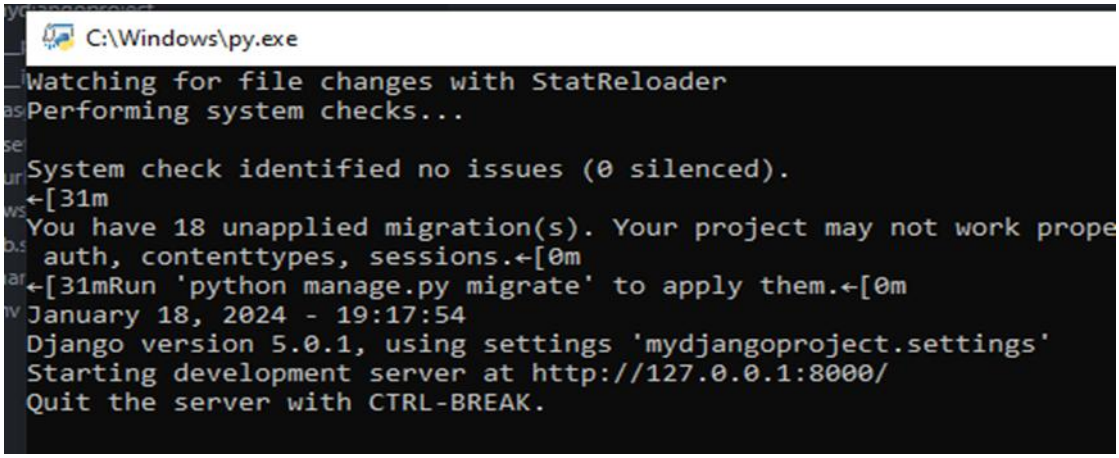



Рисунок 22 - Запущенный локальный сервер по адресу http://127.0.0.1:8000/

Если Вы заметили, то в консоли красными буквами появилось предупреждение, что имеется 18 несозданных миграций.

**Миграция** – процесс автоматического обновления базы данных при изменении модели приложения.

Выполним миграции командой ./manage.py migrate (рисунок 23).

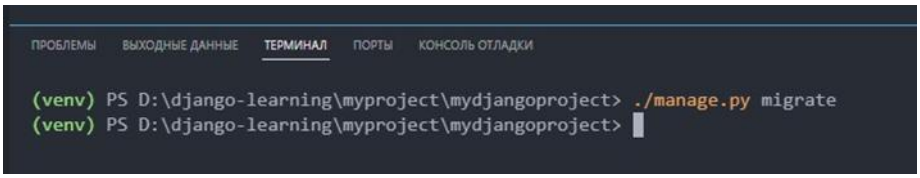



Рисунок 23 - Выполнение миграций командой ./manage.py migrate

После того, как мы пропишем данную команду и нажмем на Enter, то у нас начнут выполняться миграции (рисунок 24).

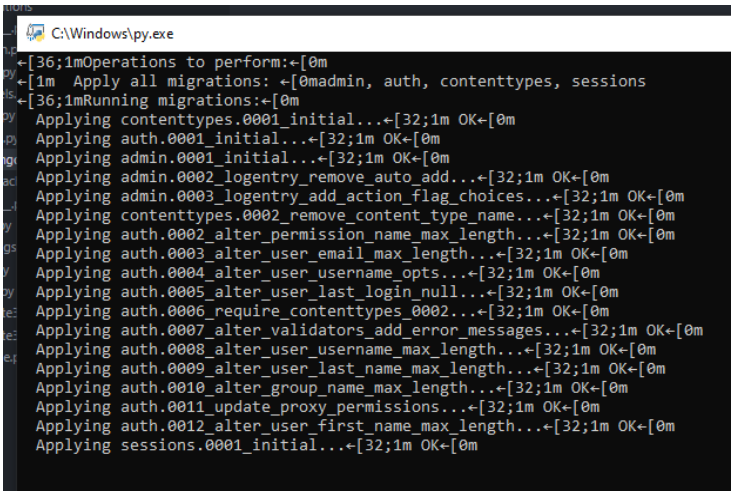



Рисунок 24 - Выполнение миграций

После выполнения миграций, снова запустим проект и перейдем по ссылке локального сервера (можно зажать Ctrl и нажать на ссылку) (рисунок 25).

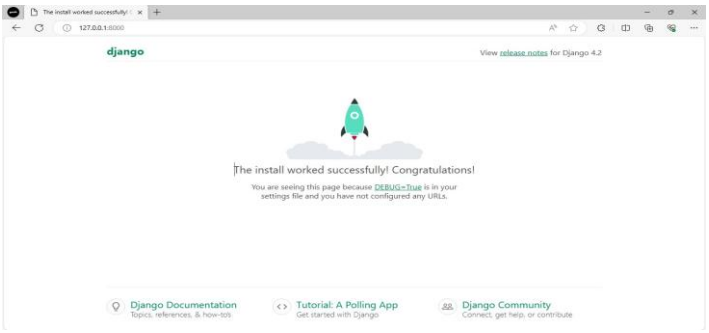



Рисунок 25 - Запущенный локальный сервер в браузере

## 6 Создание Django-приложения (Django-app)

**Django-app** – модуль, который выполняет конкретные функции в рамках веб- приложения. Каждое приложение включает в себя свою структуру URL-адресов, модели данных, представления (views), шаблоны и статические файлы. Django- app’s логически разделяют функциональность веб-приложения на более управляемые компоненты (рисунок 26).

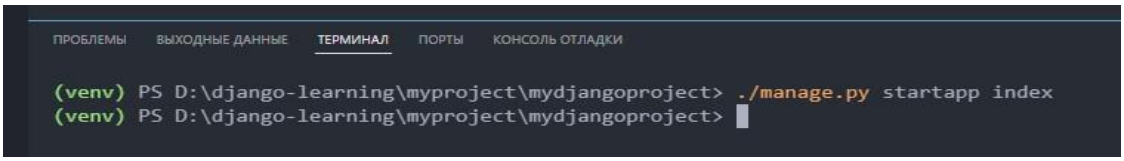



Рисунок 26 - Команда для создания Django-прлиожения

В папке Django-проекта мы видим изменение. Добавилась папка index. В ней мы видим кучу новых файлов, но обо всем по порядку (рисунок 27).

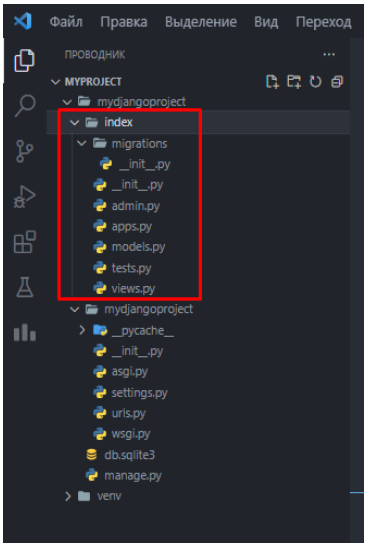



Рисунок 27 Папка Django-приложения (index)

Структура Django-приложения включает следующие основные элементы (таблица 1):

Таблица 1 - Структура приложения

| **Структура приложения** | |
|---|---|
| Папка migrations | Создание файлов миграции |
| admin.py | Регистрация таблиц в БД для админов |
| apps.py | Информация о приложении |
| models.py | Создание таблиц для БД |
| tests.py | Создание тестов для проекта |
| views.py | Создание функций, для отображения шаблонов на сайте |

7 Осуществим **Регистрацию приложения** в файле настроек проекта.

Заходим в файл settings.py и в списке констант INSTALLED_APPS указываем index, для того, что зарегистрировать наше веб-приложение (рисунок 28).

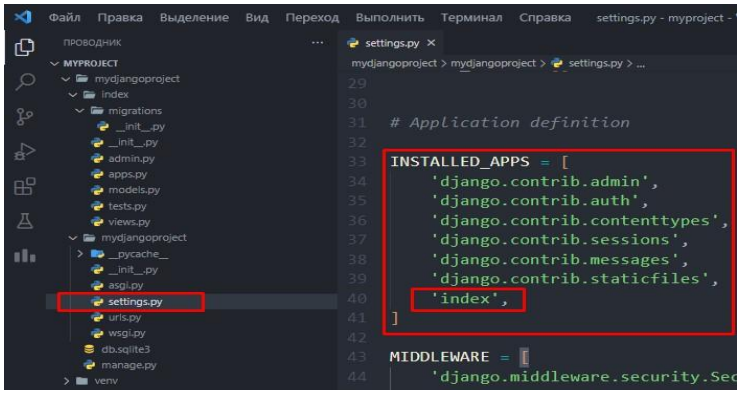



Рисунок 28 - Регистрация Django-приложения в INSTALLED_APPS

Создадим папку templates (файлы шаблонов в папке приложения)

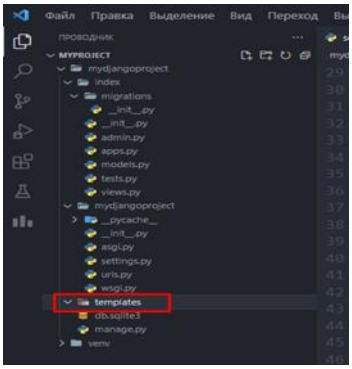



Рисунок 28 - Папка templates

После создании папки для хранения файлов шаблонов веб-приложения, её необходимо зарегистрировать в нашем проекте.

8 Регистрация папки templates

За конфигурацию шаблонов в проекте отвечает переменная TEMPLATES в файле settings.py. Укажем в «DIRS» в квадратных скобках (в списке) строку templates (рисунок 29).

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': ['templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Рисунок 29 - Регистрация папки templates в TEMPLATES

Из рекомендаций: для регистрации папки templates лучше всего использовать константу директории TEMPLATE_DIR и передавать её в ключ «DIRS» (рисунок 30).

```
TEMPLATE_DIR = os.path.join(BASE_DIR, 'templates')
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [TEMPLATE_DIR],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Рисунок 30 - Создание константы директории TEMPLATE_DIR

9 Создание файла index.html.

Далее, создадим файл index.html в папке templates (рисунок 31).

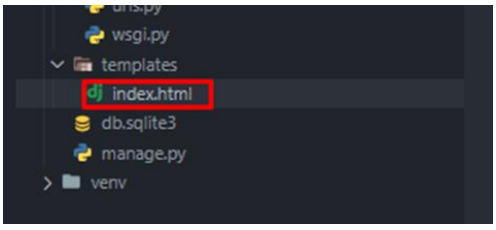



Рисунок 31 - Файл index.html в папке templates

Пропишем в данном файле небольшой код на языке гипертекстовой разметки HTML, который будет выводить в теге заголовка первого уровня текст «Главная страница» и в теге параграфа текст «Это главная страница нашего веб-приложения» (рисунок 32).

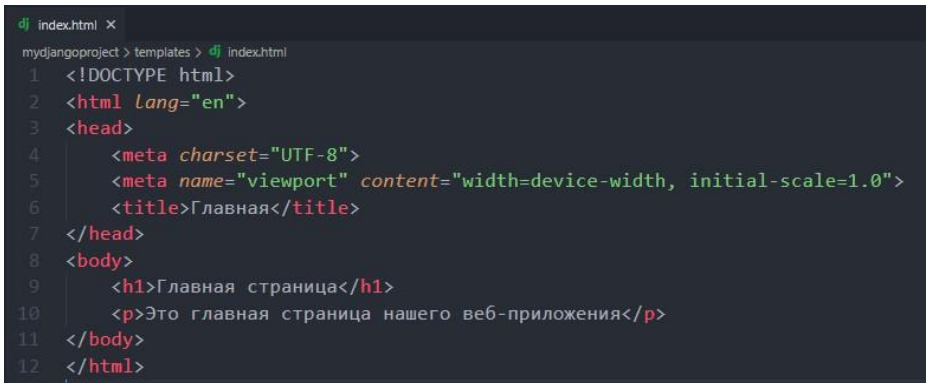



Рисунок 32 - Код для файла index.html

10 Создание функции в views.py для открытия главной страницы. Создадим функцию для открытия страницы в файле views.py (рисунок 33).

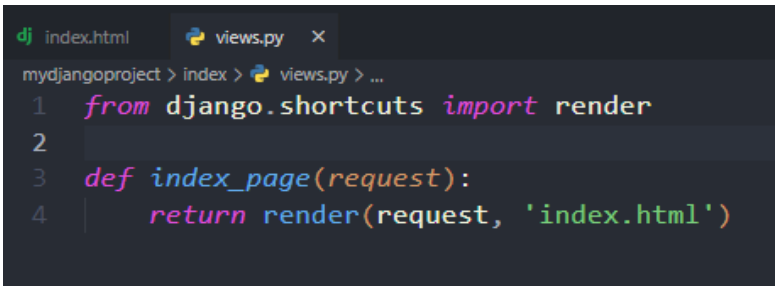



Рисунок 33 - Функция index_page для обработки страницы index.html

Здесь, немного остановимся и давайте поймем, что мы написали. Мы определяем функцию с именем **index_page**, которая принимает аргумент request. В Django **каждая функция представления** принимает объект **request**, представляющий запрос, отправленный клиентом (веб-браузером).

return render(request, 'index.html')

Эта строка означает, что при обращении к представлению index_page, мы хотим **вернуть HTML-страницу**, которая будет отображаться пользователю.

Функция **render** отвечает за обработку запроса и генерацию HTML-страницы на основе шаблона.

11 Определим маршрут в файле urls.py для созданной view-функции (рисунок 34).

```
17  from django.contrib import admin
18  from django.urls import path
19  from index.views import index_page
20
21  urlpatterns = [
22      path('admin/', admin.site.urls),
23      path('', index_page)
24  ]
```

Рисунок 34 - Определение маршрута для главной страницы

Остановимся на этом этапе и поймем, что мы сделали.

– **urlpatterns** – переменная, которая содержит список URL-адресов и соответствующих обработчиков представлений;

– **path('admin/', admin.site.urls)** – данный путь веб-сайта связан с админ панелью. Когда пользователь переходит по адресу, который начинается с '/admin/', Django будет направлять запрос на соответствующий обработчик для административной панели;

– **path('', index_page)** – данный путь веб-сайта связан с обработчиком представления index_page. Когда пользователь посещает корневой URL (например, http://example.com/), Django будет направлять запрос на функцию index_page;

– **from index.views import index_page** – из файла **views**, который находится в папке приложения **index** мы импортируем функцию **index_page**.

12 Запустим Django-проект (рисунок 35)!

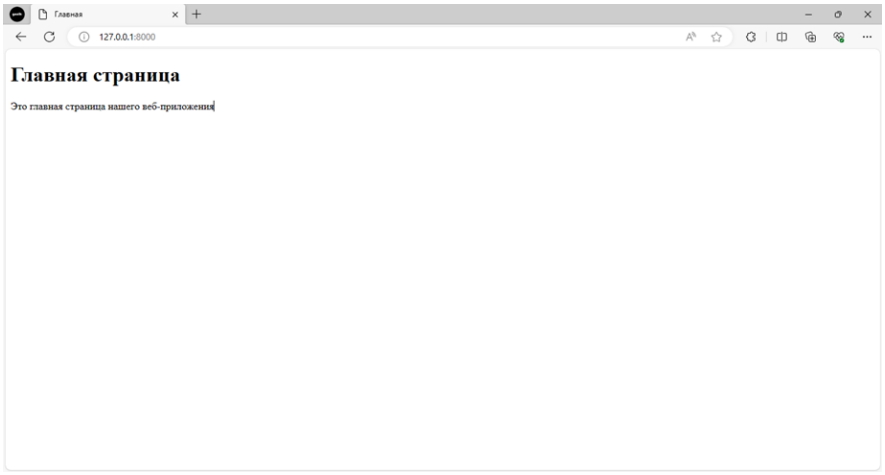



Рисунок 35 - Запущенный локальный сервер проекта

### 4.6 Пример решения задачи на языке программирования Python

Задача. Создайте веб-проект на фреймворке Django с именем **blog_site**. Внутри проекта создайте приложение **posts**, предназначенное для работы с

публикациями блога. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

**Решение задачи:**

1 Создание проекта

```
django-admin startproject blog_site
cd blog_site
```

2 Создание приложения

```
python manage.py startapp posts
```

3 Подключение приложения

Откройте файл blog_site/settings.py и добавьте приложение в INSTALLED_APPS:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'posts',  # добавленное приложение
]
```

4 Запуск сервера

```
python manage.py runserver
```

5 Проверка

Откройте в браузере:

http://127.0.0.1:8000/

## 5 Индивидуальное задание

5.1 Установите и настройте фреймворк Django на компьютер и проверьте установленную версию фреймворка.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте веб-проект на фреймворке Django с именем library_site. Внутри проекта создайте приложение books, предназначенное для работы с информацией о книгах. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

2 Создайте веб-проект на фреймворке Django с именем student_portal. Внутри проекта создайте приложение students, предназначенное для работы с

информацией о студентах. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

3 Создайте веб-проект на фреймворке Django с именем movie_portal. Внутри проекта создайте приложение films, предназначенное для работы с информацией о фильмах. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

4 Создайте веб-проект на фреймворке Django с именем music_library. Внутри проекта создайте приложение albums, предназначенное для работы с информацией о музыкальных альбомах. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

5 Создайте веб-проект на фреймворке Django с именем online_shop. Внутри проекта создайте приложение products, предназначенное для работы с информацией о товарах интернет-магазина. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

6 Создайте веб-проект на фреймворке Django с именем news_portal. Внутри проекта создайте приложение news, предназначенное для работы с новостями. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

7 Создайте веб-проект на фреймворке Django с именем tourism_portal. Внутри проекта создайте приложение tours, предназначенное для работы с информацией о туристических маршрутах. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

8 Создайте веб-проект на фреймворке Django с именем sport_club_site. Внутри проекта создайте приложение teams, предназначенное для работы с информацией о спортивных командах. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

9 Создайте веб-проект на фреймворке Django с именем school_portal. Внутри проекта создайте приложение teachers, предназначенное для работы с информацией о преподавателях. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер

разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

10 Создайте веб-проект на фреймворке Django с именем hospital_system. Внутри проекта создайте приложение patients, предназначенное для работы с информацией о пациентах. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

11 Создайте веб-проект на фреймворке Django с именем education_platform. Внутри проекта создайте приложение courses, предназначенное для работы с учебными курсами. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

12 Создайте веб-проект на фреймворке Django с именем restaurant_site. Внутри проекта создайте приложение menu, предназначенное для работы с меню ресторана. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

13 Создайте веб-проект на фреймворке Django с именем car_catalog. Внутри проекта создайте приложение cars, предназначенное для работы с информацией об автомобилях. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

14 Создайте веб-проект на фреймворке Django с именем portfolio_site. Внутри проекта создайте приложение main, предназначенное для работы с основной информацией сайта. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

15 Создайте веб-проект на фреймворке Django с именем home_site. Внутри проекта создайте приложение home, предназначенное для работы с основной информацией сайта. Добавьте созданное приложение в список INSTALLED_APPS в файле settings.py. После этого запустите встроенный сервер разработки Django и убедитесь, что проект успешно запускается и открывается в браузере по адресу http://127.0.0.1:8000.

**6 Содержание отчета** (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

## 7 Контрольные вопросы

7.1 Опишите, что такое фреймворк Django в языке программирования Python.

7.2 Охарактеризуйте, что представляет собой модель (Model) в Django в языке программирования Python.

7.3 Объясните, какую роль выполняет представление (View) в Django в языке программирования Python.

7.4 Опишите, для чего используются шаблоны (Templates) в Django в языке программирования Python.

7.5 Укажите, с помощью какой команды устанавливается Django в языке программирования Python.

7.6 Укажите, какая команда используется для создания проекта Django в языке программирования Python.

7.7 Опишите, как создать приложение внутри проекта Django в языке программирования Python.

## Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.